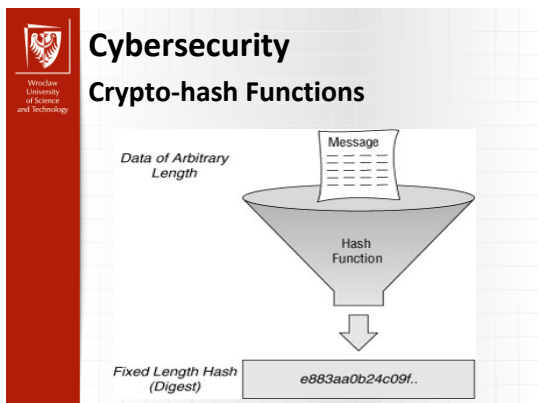




Cybersecurity

Agenda

- Non-crypto hash functions
- Crypto hash function – properties
- Digital Signatures
- Signature Birthday Attack

Cybersecurity

Crypto Hash Function Design

- Desired property: **avalanche effect**
 - Any change to input affects lots of output bits
- Crypto hash functions consist of some number of rounds
 - Analogous to block cipher in CBC mode
- Want security and speed
 - Avalanche effect after few rounds
 - But simple rounds

Cybersecurity

Crypto Hash Function Design

- Input data split into blocks
- **Compression function** applied to blocks
 - Current block and previous block output
 - Output for last block is the hash value
- For hashes we consider
 - Block size is 512 bits
 - Compression function output is 128 bits

Cybersecurity

Crypto Hash Function Design

- Input or “message” blocks $M_0, M_1, \dots, M_{N-1}$
- Addition is mod 2^{32} per 32-bit word
- This is known as **Merkle-Damgard construction**

IV

M_0 — 512

128

compression function

M_1 — 512

128

compression function

M_{N-1} — 512

128

compression function

hash value

Cybersecurity

Crypto Hash

- If msg is one 512-bit block: $h(M) = f(IV, M)$ where f and IV known to Trudy
- For 2 blocks:

$$h(M) = f(f(IV, M_0), M_1) = f(h(M_0), M_1)$$
- In general $h(M) = f(h(M_0, M_1, \dots, M_{n-2}), M_{n-1})$
 - If $h(M) = h(M')$ then $h(M, X) = h(M', X)$ for any X
 - Implications for design of “hashed MAC”...



Cybersecurity

HMAC

- MAC: block cipher for integrity
- Can we use a hash function instead?
- A “hashed MAC”, **HMAC**, of M with key K
 - Why is a key necessary?
- How to compute HMAC?
- Two obvious choices: $h(K,M)$ and $h(M,K)$
- Which (if either) is better?



Cybersecurity

HMAC

- Should we compute HMAC as $h(K,M)$?
- Hashes computed in blocks
- Recall $h(M_0, M_1) = F(h(M_0), M_1)$
- Let $M' = (M, X)$
 - Then $h(K, M') = F(h(K, M), X)$
 - Trudy can compute HMAC of M' without K
 - Defeats the purpose of HMAC



Cybersecurity

HMAC

- Should we compute HMAC as $h(M,K)$?
 - Is this better than $h(K,M)$?
- If $h(M') = h(M)$ then

$$h(M,K) = F(h(M), K) = F(h(M'), K) = h(M', K)$$
- In this case, Trudy can compute HMAC without knowing the key K
 - But collision must be known
 - Better than $h(K,M)$, but we can do better



Cybersecurity

HMAC Function

Hash-based Message Authentication Code – a cryptographic function that authenticates both the **integrity** and **authenticity** of a message.

Definition

$$\text{HMAC}(K, M) = H[(K' \oplus \text{opad}) \parallel H[(K' \oplus \text{ipad}) \parallel M]]$$

where:

K = secret key | M = message | H = hash function (SHA-256, SHA-512, etc.) |
ipad = 0x36 repeated | opad = 0x5C repeated

Key Properties

- **Two-pass hashing:** Inner hash with message → Outer hash with result
- **Symmetric key:** Sender and receiver share the same secret key
- **Fast & secure:** Immune to length-extension attacks, millions of verifications/sec



Cybersecurity

Hashing and Birthdays

- The “birthday problem” arises in many crypto contexts
- We discuss it in hashing context
 - And “birthday attack” on digital signature



Cybersecurity

Pre-Birthday Problem

- Suppose N people in a room
- How large must N be before the probability someone has the same birthday as me is $\geq 1/2$
 - Solve:

$$1/2 = 1 - (364/365)^N \text{ for } N$$
 - Find:

$$N = 253$$



Cybersecurity

Birthday Problem

- How many people must be in a room before probability is $\geq 1/2$ that two or more have same birthday?
 - $1 - 365/365 \cdot 364/365 \cdot \dots \cdot (365-N+1)/365$
 - Set equal to $1/2$ and solve: **$N = 23$**
- Surprising? A paradox?
- No, it "should be" about $\sqrt{365}$ since compare **pairs** x and y



Cybersecurity

Birthday Problem

- If $h(x)$ is N bits, 2^N hash values
- Note that $\sqrt{2^N} = 2^{N/2}$
- So, hash $2^{N/2}$ inputs and find a collision
 - "Birthday attack" — an exhaustive search
- An N -bit symmetric cipher key requires at most 2^{N-1} work to "break"
- An N -bit hash: at most $2^{N/2}$ work to "break"



Cybersecurity

Signature Birthday Attack

- Suppose hash output is n bits
- Trudy selects evil message E
 - Wants to get Alice's signature on E
- Trudy creates innocent message I
 - Alice willing to sign message I
- How can Trudy use birthday problem?



Cybersecurity

Signature Birthday Attack

- Trudy creates $2^{n/2}$ variants of I
 - All have same meaning as I
 - Trudy hashes each: $h(I_0), h(I_1), \dots$
- Trudy creates $2^{n/2}$ variants of E
 - All have same meaning as E
 - Trudy hashes each: $h(E_0), h(E_1), \dots$
- By birthday problem, $h(I_j) = h(E_k)$, some j, k



Cybersecurity

Signature Birthday Attack

- Alice signs innocent message I_j
- Then Trudy has $[h(I_j)]_{\text{Alice}}$
- But $[h(I_j)]_{\text{Alice}} = [h(E_k)]_{\text{Alice}}$
- Alice unwittingly "signed" evil E_k
- Attack relies only on birthday problem



Cybersecurity

Online Bid Example

- Suppose Alice, Bob, Charlie are bidders
- Alice plans to bid A , Bob B and Charlie C
 - They do not trust that bids will be secret
 - Nobody willing to submit their bid
- Solution?
 - Alice, Bob, Charlie submit **hashes** $h(A), h(B), h(C)$
 - All hashes received and posted online
 - Then bids A, B and C revealed
- Hashes do not reveal bids (one way)
- Cannot change bid after hash sent (collision)



Cybersecurity

Online Bid Example

- This protocol is not secure!
- A forward search attack is possible
 - Bob computes $h(A)$ for likely bids A
- How to prevent this?
- Alice computes $h(A,R)$, R is random
 - Then Alice must reveal A and R
 - Trudy cannot try all A and R



Cybersecurity

Online Bid Example

- Spse $B = \$1$ and Bob submits $h(B,R)$
- When revealed, $A = \$1$ and $C = \$2$
- Bob wants to change his bid: $B' = \$3$
- Bob computes $h(B',R')$ for different R' until he finds $h(B',R') = h(B,R)$
 - How much work?
 - Apparently, about 2^n hashes required



Cybersecurity

Spam Reduction

- Spam reduction
(spam - *unsolicited or undesirable electronic messages*)
- Before I accept an email from you, I **want proof that you spent "effort"** (e.g., CPU cycles) to create the email
- Limit amount of email that can be sent
- Make spam much more costly to send



Cybersecurity

Spam Reduction

- Let M = email message
- Let R = value to be determined
- Let T = current time
- Sender must find R such that
 - $\text{hash}(M, R, T) = (00 \dots 0, X)$, where
 - N initial bits of hash are **all zero**
- Sender then sends (M, R, T)
 - Recipient accepts email, provided
 - $\text{hash}(M, R, T)$ begins with N zeros



Cybersecurity

Spam Reduction

- **Sender:** $\text{hash}(M, R, T)$ begins with N zeros
- **Recipient:** verify that $\text{hash}(M, R, T)$ begins with N zeros
- **Work for sender:** about 2^N hashes
- **Work for recipient:** 1 hash
- Sender's **work increases exponentially** in N
- Same **work for recipient regardless** of N
- Choose N so that
 - Work acceptable for normal email users
 - Work unacceptably high for spammers!



Cybersecurity

Public Key Notation

- **Sign** message M with Alice's **private key**: $[M]_{\text{Alice}}$
- **Encrypt** message M with Alice's **public key**: $\{M\}_{\text{Alice}}$
- Then
 - $\{[M]_{\text{Alice}}\}_{\text{Alice}} = M$
 - $[\{M\}_{\text{Alice}}]_{\text{Alice}} = M$

Cybersecurity

Digital Signatures

- Suppose Alice signs M
 - Alice sends M and $S = [M]_{\text{Alice}}$ to Bob
 - Bob verifies that $M = \{S\}_{\text{Alice}}$
- If M is big, $[M]_{\text{Alice}}$ is costly to compute
- Suppose instead, Alice signs $h(M)$, where $h(M)$ is much smaller than M
 - Alice sends M and $S = [h(M)]_{\text{Alice}}$ to Bob
 - Bob verifies that $h(M) = \{S\}_{\text{Alice}}$

Cybersecurity

Digital Signatures

- Digital signatures provide **integrity**
 - Like MAC and HMAC
- Why?
 - Alice sends M and $S = [h(M)]_{\text{Alice}}$ to Bob
 - If M changed to M' or S changed to S' (accident or intentional) Bob detects it:

$$h(M') \neq \{S\}_{\text{Alice}}, h(M) \neq \{S'\}_{\text{Alice}}, h(M') \neq \{S'\}_{\text{Alice}}$$

Cybersecurity

Non-repudiation

- Digital signature also provides for **non-repudiation**
- Alice sends M and $S = [h(M)]_{\text{Alice}}$ to Bob
- Alice cannot "repudiate" signature
 - Alice cannot claim she did not sign M
- Why does this work?

Cybersecurity

Hashing and Signatures

- Alice signs $h(M)$, sends M and $S = [h(M)]_{\text{Alice}}$ to Bob and Bob verifies $h(M) = \{S\}_{\text{Alice}}$
- Security depends on public key system **and** hash function
- Suppose Trudy can find collision: $M' \neq M$ with $h(M') = h(M)$
- Then Trudy can replace M with M' and **signature scheme is broken**

Cybersecurity

Crypto Hash Functions

